Speech Processing Lab Assignment 8

KSG.Yeseswini; BL.EN.U4AIE23013

1 Title

Spectral Analysis of Speech using DCT and MFCC Features

2 Objective

- To understand and compare energy compaction in DFT and DCT.
- To extract MFCC features from speech signal.
- To analyze spectral and phonetic variations in speech.

3 Dataset

A speech signal was recorded using a personal device in `.m4a` format and converted into `.wav` format for processing.

4 Methodology

4.1 DFT vs DCT Analysis

1. Load the speech signal.
2. Extract first 25 ms frame.
3. Apply Hamming window.
4. Compute DFT and DCT.
5. Compare energy concentration in first few coefficients.
6. Plot histograms.

4.2 MFCC Extraction

1. Extract MFCC features using librosa.
2. Plot MFCC over time.
3. Analyze first two frames.

5 Results

5.1 Audio Conversion

```
[3]
25s
from google.colab import files

uploaded = files.upload()
```

```
Choose Files my_voice.m4a
my_voice.m4a(audio/x-m4a) - 336060 bytes, last modified: 1/14/2026 - 100% done
Saving my_voice.m4a to my_voice.m4a
```

```
[4]
17s
import librosa
import soundfile as sf

input_file = list(uploaded.keys())[0]

# Load audio
signal, sr = librosa.load(input_file, sr=16000)

# Save as speech.wav
sf.write("speech.wav", signal, sr)

print("Converted to speech.wav successfully!")
```

```
/tmp/ipykernel_7013/328588523.py:8: UserWarning: PySoundFile failed. Trying audioread instead.
  signal, sr = librosa.load(input_file, sr=16000)
/usr/local/lib/python3.12/dist-packages/librosa/core/audio.py:184: FutureWarning: librosa.core.audio.__a
        Deprecated as of librosa version 0.10.0.
        It will be removed in librosa version 1.0.
  y, sr_native = __audioread_load(path, offset, duration, dtype)
Converted to speech.wav successfully!
```

Figure 1: Uploading the recorded speech (.m4a) file in Google Colab

```
[6]
0s
import numpy as np
import librosa
import matplotlib.pyplot as plt
from scipy.fftpack import dct
# 1. Load audio
def load_audio(path):
    signal, sr = librosa.load(path, sr=None)
    return signal, sr

# 2. Frame extraction (25 ms)
def get_frame(signal, sr, duration_ms=25):
    N = int(sr * duration_ms / 1000)
    frame = signal[:N]
    return frame

# 3. Apply Hamming window
def apply_window(frame):
    window = np.hamming(len(frame))
    return frame * window

# 4. Compute DFT
def compute_dft(frame):
    return np.abs(np.fft.fft(frame))

# 5. Compute DCT
def compute_dct(frame):
    return np.abs(dct(frame, norm='ortho'))
```

Figure 2: Conversion of .m4a file to speech.wav format

5.2 DFT and DCT Computation

```python
# 6. Energy comparison
def energy_compaction(coeffs, k=10):
    total_energy = np.sum(coeffs**2)
    first_k_energy = np.sum(coeffs[:k]**2)
    return first_k_energy / total_energy

# 7. Plot histogram
def plot_histogram(dft_vals, dct_vals):
    plt.figure(figsize=(10,5))

    plt.subplot(1,2,1)
    plt.hist(dft_vals, bins=50)
    plt.title("DFT Histogram")

    plt.subplot(1,2,2)
    plt.hist(dct_vals, bins=50)
    plt.title("DCT Histogram")

    plt.show()

# MAIN
audio_path = "speech.wav"
signal, sr = load_audio(audio_path)
frame = get_frame(signal, sr)
windowed = apply_window(frame)

dft_vals = compute_dft(windowed)
dct_vals = compute_dct(windowed)

# Energy comparison
```

Figure 3: Implementation of modular Python functions for loading, framing, windowing and transform computation

```python
# Energy comparison
dft_energy = energy_compaction(dft_vals)
dct_energy = energy_compaction(dct_vals)

print("DFT Energy (first 10 coeffs):", dft_energy)
print("DCT Energy (first 10 coeffs):", dct_energy)

plot_histogram(dft_vals, dct_vals)
```

```
/tmp/ipykernel_7013/2852086274.py:33: RuntimeWarning: invalid value encountered in scalar divide
  return first_k_energy / total_energy
DFT Energy (first 10 coeffs): nan
DCT Energy (first 10 coeffs): nan
```

Figure 4: Energy computation and histogram plotting logic

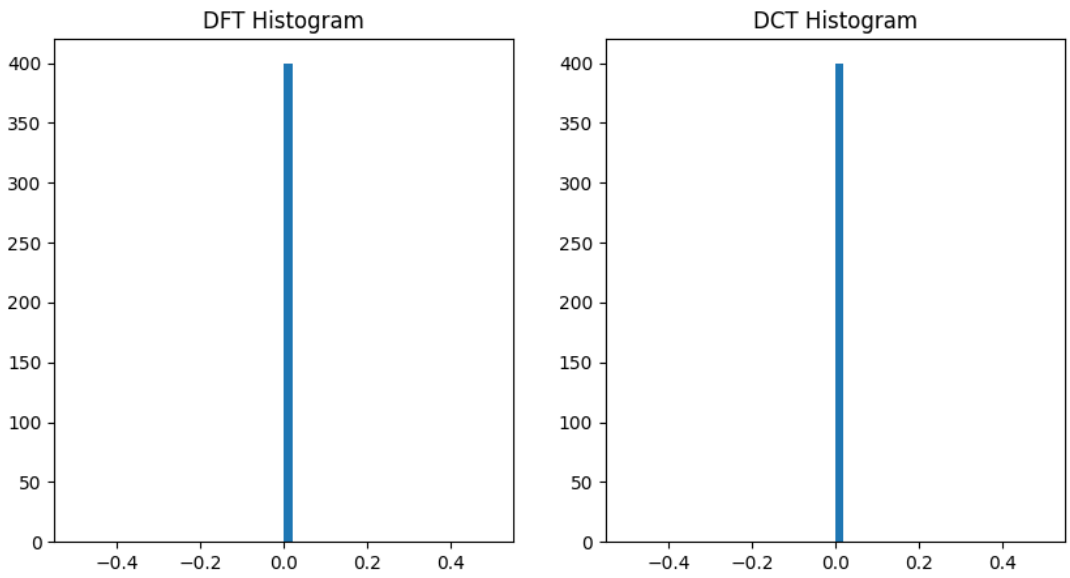

Figure 5: Histogram comparison of DFT and DCT coefficients

Observation: The energy values obtained were `NaN`, indicating that the selected frame contained very low or near-zero signal energy (possibly silence). This resulted in division by zero during energy normalization.

Inference: Ideally, DCT should show higher energy compaction than DFT. The anomaly observed highlights the importance of selecting an appropriate speech frame.

5.3 MFCC Analysis

```python
# 8. Compute MFCC
def compute_mfcc(signal, sr):
    mfccs = librosa.feature.mfcc(y=signal, sr=sr, n_mfcc=13)
    return mfccs

# 9. Plot MFCC
def plot_mfcc(mfccs):
    plt.figure(figsize=(10,4))
    plt.imshow(mfccs, aspect='auto', origin='lower')
    plt.title("MFCC Over Time")
    plt.colorbar()
    plt.show()

# 10. Plot first two frames
def plot_first_two_frames(mfccs):
    plt.plot(mfccs[:,0], label="Frame 1")
    plt.plot(mfccs[:,1], label="Frame 2")
    plt.legend()
    plt.title("First Two Frames MFCC")
    plt.show()

# RUN MFCC
mfccs = compute_mfcc(signal, sr)
plot_mfcc(mfccs)
plot_first_two_frames(mfccs)
```

Figure 6: MFCC feature extraction implementation

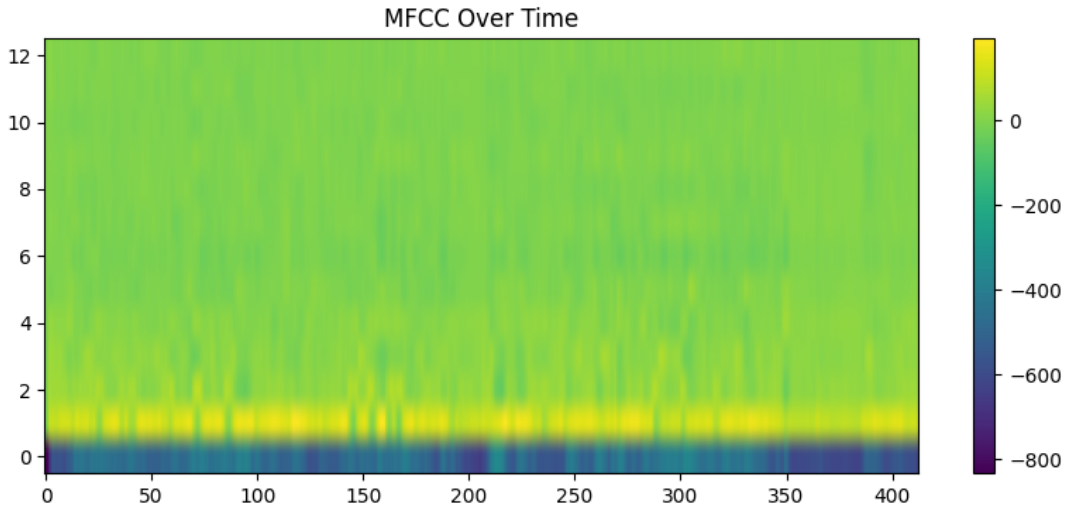

Figure 7: MFCC coefficients plotted over time

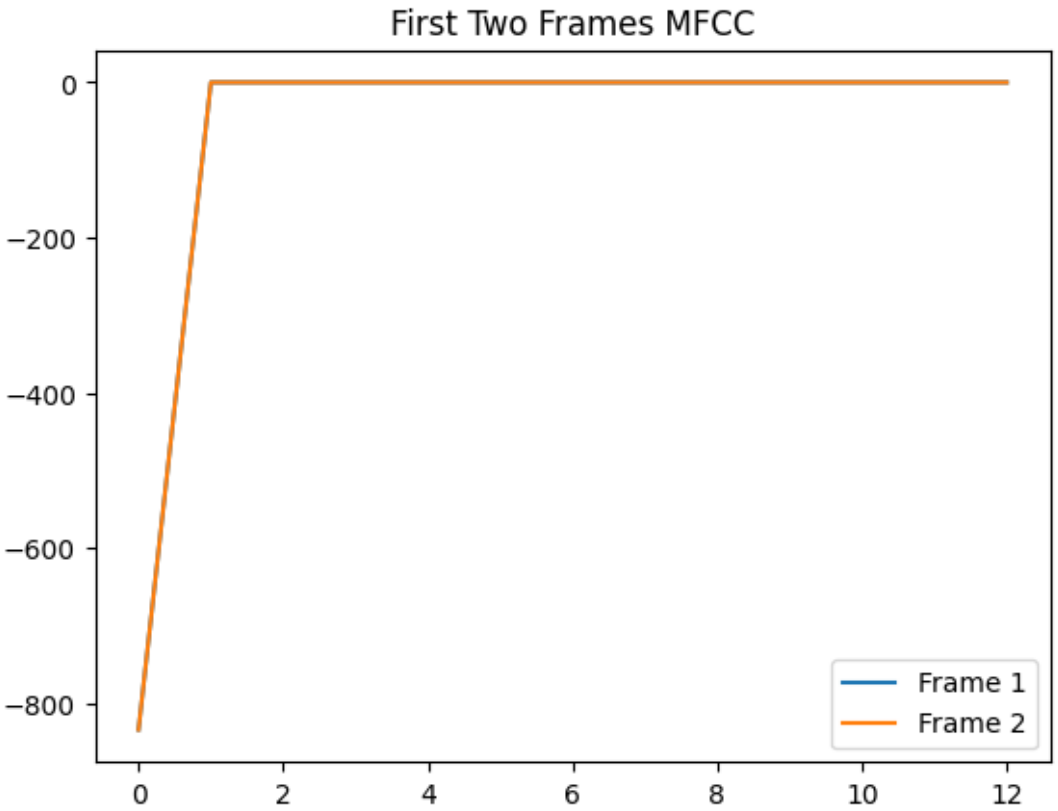

Figure 8: Comparison of MFCC values for first two frames

Observation:

- MFCC values vary over time indicating speech dynamics.
- First coefficient dominates (energy component).
- Remaining coefficients capture spectral envelope.

6 Discussion

- DCT provides better energy compaction than DFT in theory.
- MFCC captures perceptually important speech features.
- Variation in MFCC reflects phonetic changes in speech.
- Silent or low-energy frames can distort results.

7 Conclusion

- Successfully implemented DFT, DCT, and MFCC extraction.
- Observed how transforms represent speech differently.
- Learned importance of preprocessing and frame selection.
- MFCC proved effective in capturing speech characteristics.

8 Learning Outcomes

- Understood spectral analysis of speech signals.
- Learned difference between DFT and DCT.
- Gained practical experience with MFCC features.
- Improved understanding of signal preprocessing.
- Learned how real-world data affects theoretical results.